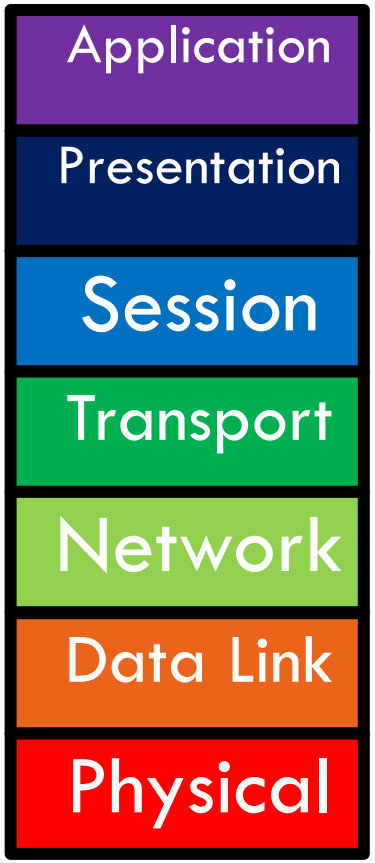
TCP: CONNECTION MANAGEMENT AND CONGESTION CONTROL

This slide is based on PPT from Northeastern University, USA

Introduction to Computer Networks
Faculty of Information Technology, KMITL
2020

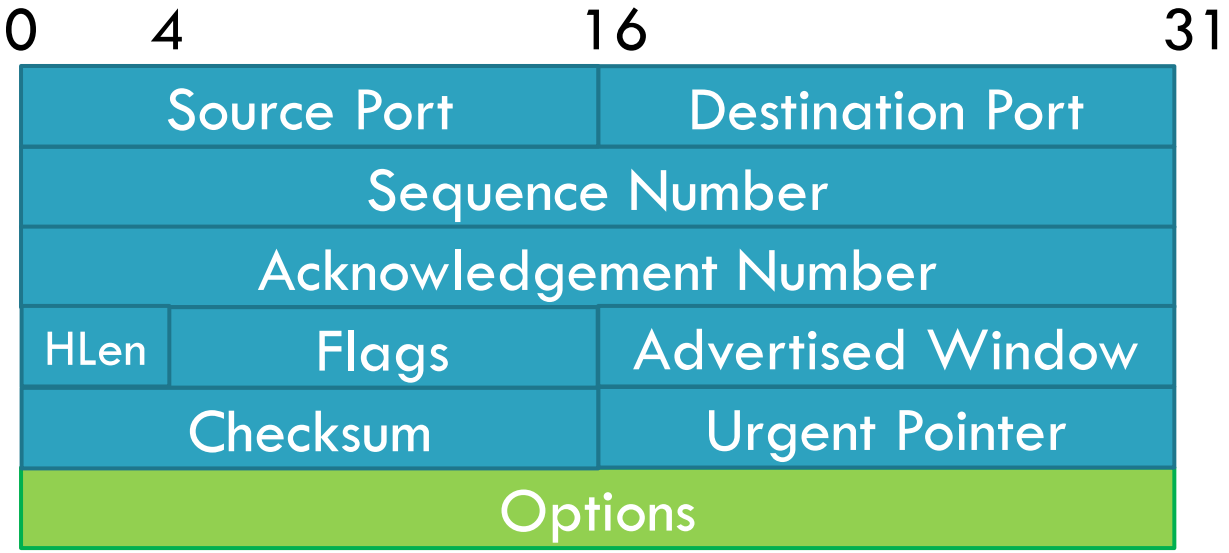
Transport Layer



- Function:
  - Demultiplexing of data streams
- Optional functions:
  - Creating long lived connections
  - Reliable, in-order packet delivery
  - Error detection
  - Flow and congestion control
- Key challenges:
  - Detecting and responding to congestion
  - Balancing fairness against high utilization

Transmission Control Protocol

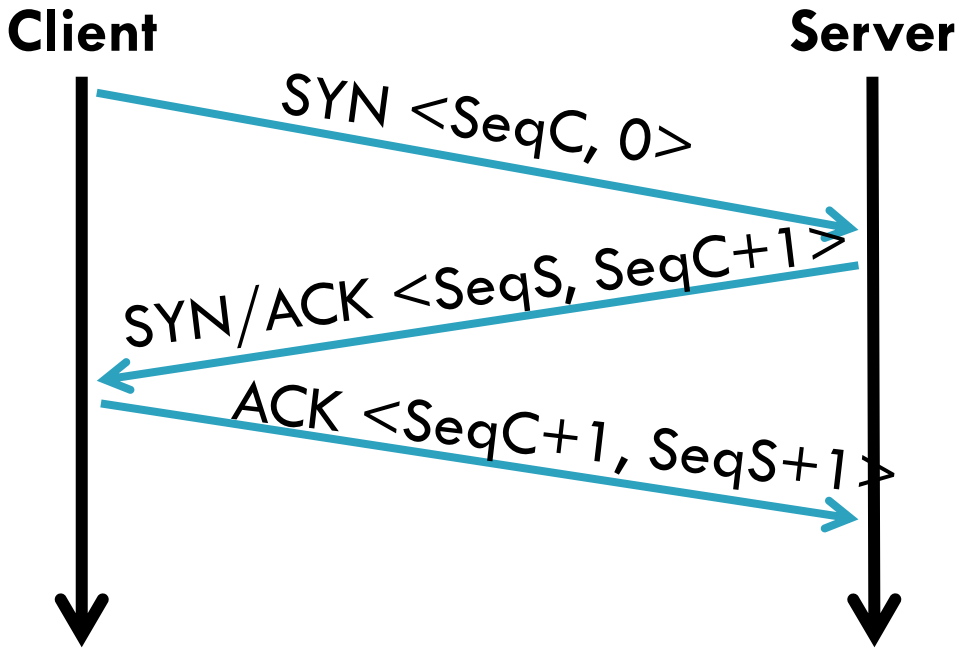
- Reliable, in-order, bi-directional byte streams
  - Port numbers for demultiplexing
  - Virtual circuits (connections)
  - Flow control
  - Congestion control, approximate fairness



Connection Setup

- Why do we need connection setup?
  - To establish state on both hosts
  - Most important state: sequence numbers
    - Count the number of bytes that have been sent
    - Initial value chosen at random
    - Why?
- Important TCP flags (1 bit each)
  - SYN – synchronization, used for connection setup
  - ACK – acknowledge received data
  - FIN – finish, used to tear down connection

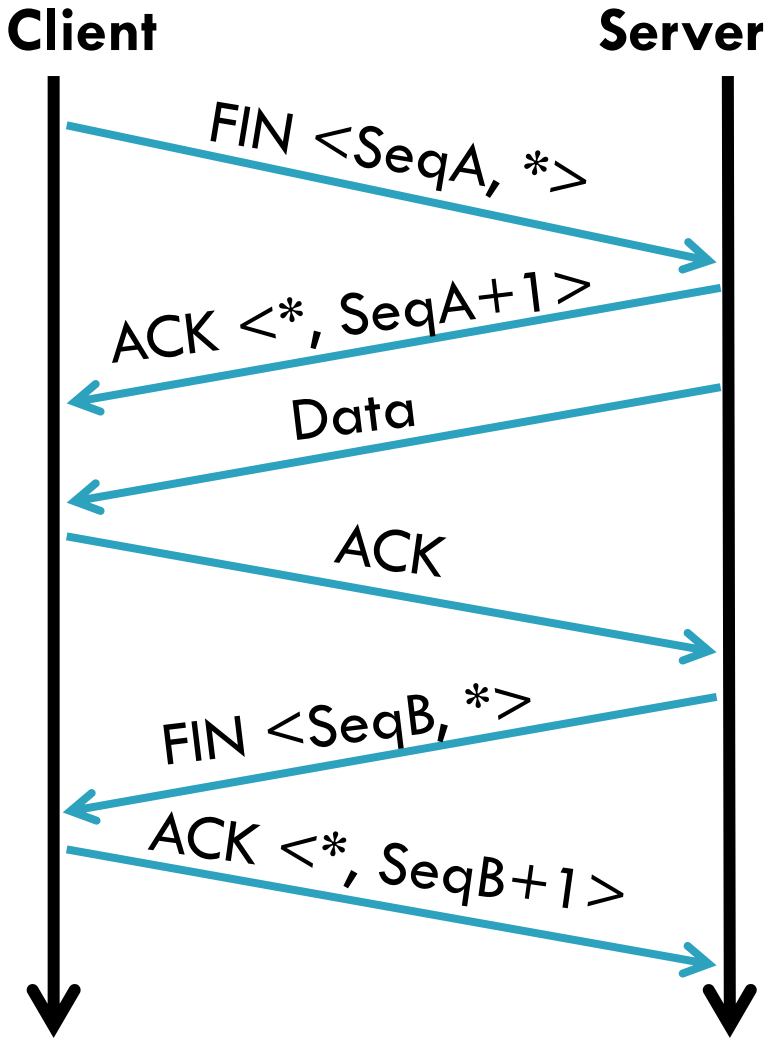
Three Way Handshake



- Each side:
  - Notifies the other of starting sequence number
  - ACKs the other side's starting sequence number (+1)

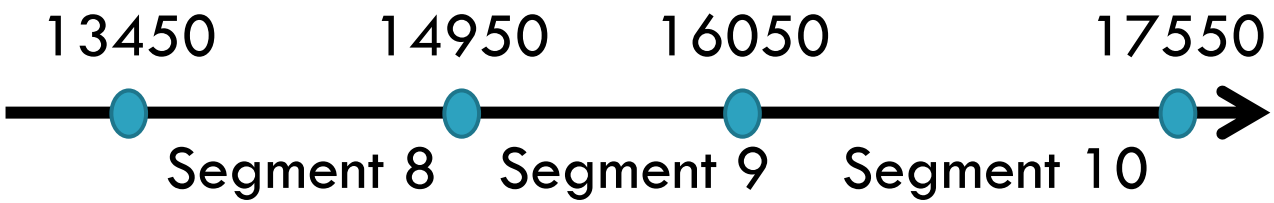
Connection Tear Down

- Either side can initiate tear down
- Other side may continue sending data
  - Half open connection
  - *shutdown()*
- Acknowledge the last FIN
  - Sequence number + 1

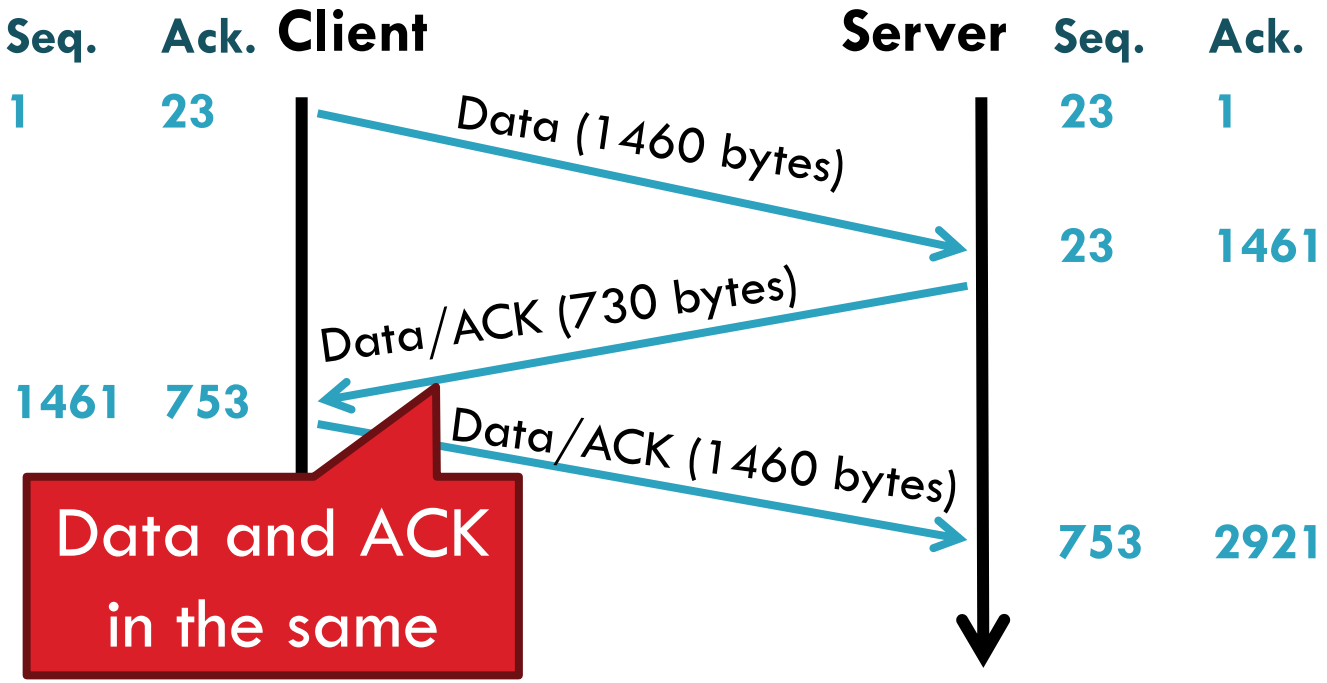


Sequence Number Space

- TCP uses a byte stream abstraction
  - Initial, random values selected during setup
  - Each byte in each stream is numbered
  - 32-bit value, wraps around
- Byte stream broken down into segments Size limited by the Maximum Segment Size (MSS, typically 1460 bytes)
  - Set to limit fragmentation
- Each segment has a sequence number

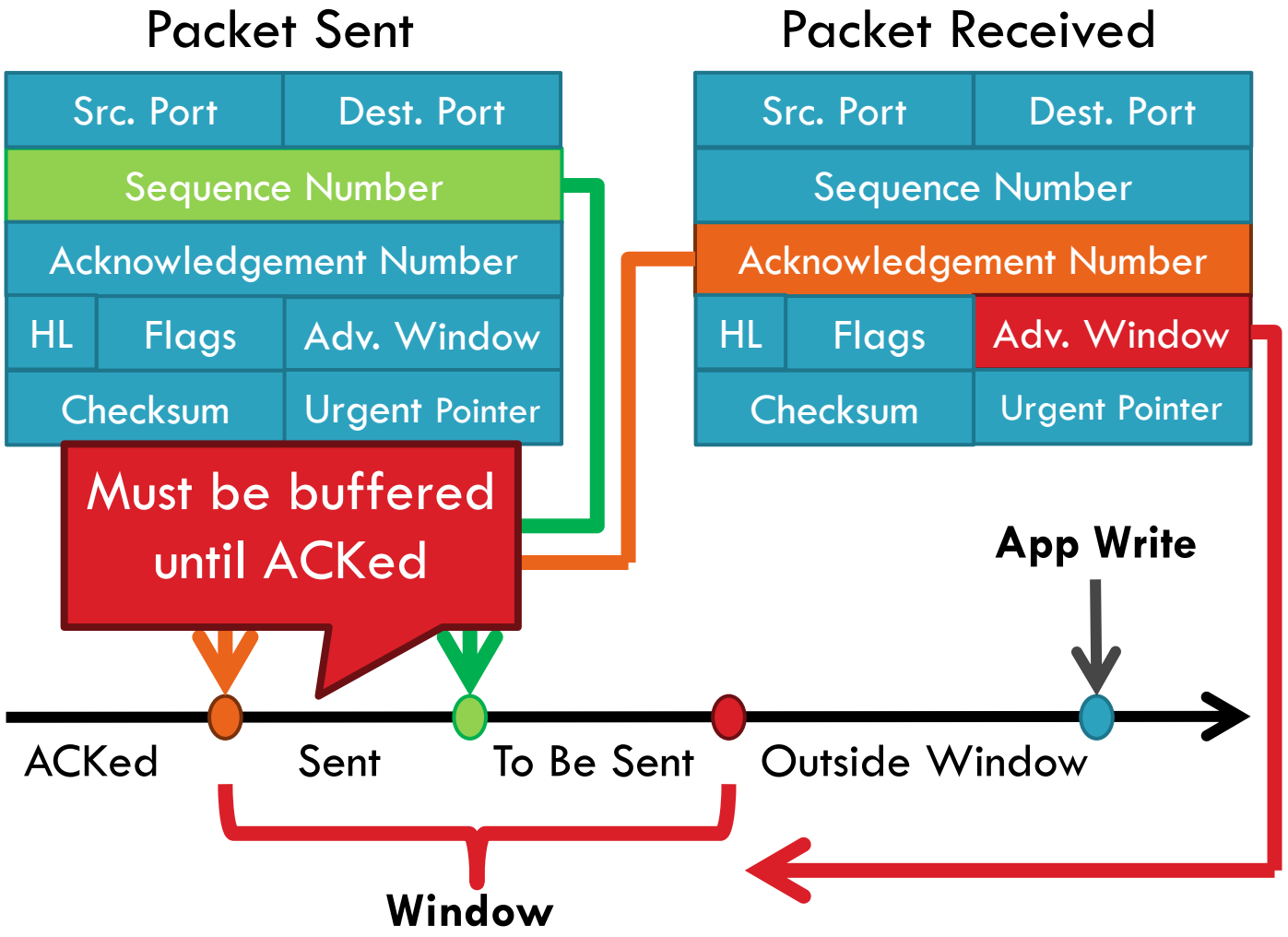


Bidirectional Communication



- Each side of the connection can send and receive
  - Different sequence numbers for each direction

Flow Control: Sender Side

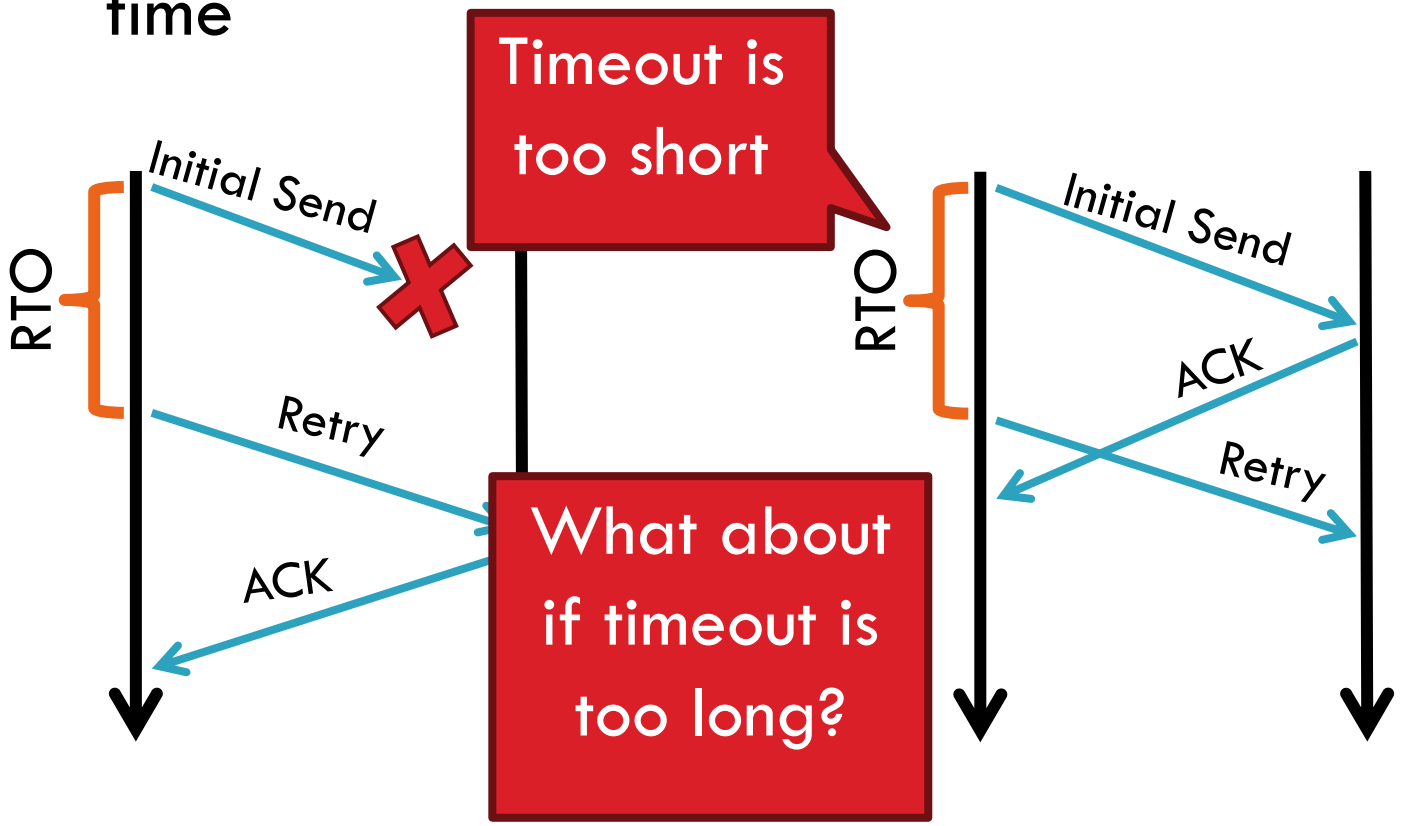


Error Detection

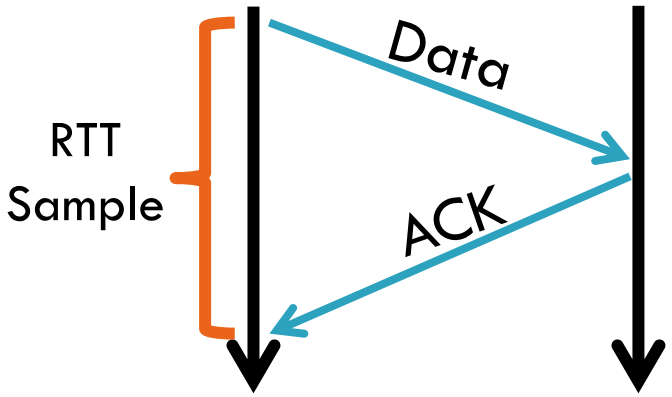
- Checksum detects (some) packet corruption
  - Computed over IP header, TCP header, and data
- Sequence numbers catch sequence problems
  - Duplicates are ignored
  - Out-of-order packets are reordered or dropped
  - Missing sequence numbers indicate lost packets
- Lost segments detected by sender
  - Use timeout to detect missing ACKs
  - Need to estimate RTT to calibrate the timeout
  - Sender must keep copies of all data until ACK

Retransmission Time Outs (RTO)

- Problem: time-out is linked to round trip time

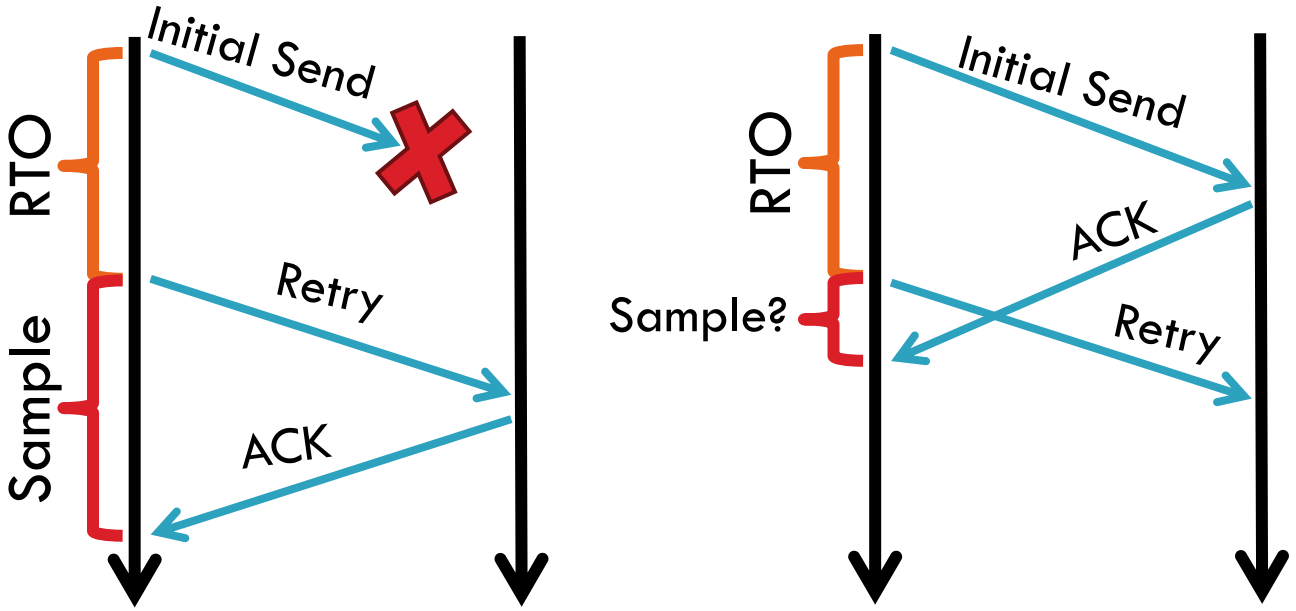


Round Trip Time Estimation



- Original TCP round-trip estimator
  - RTT estimated as a moving average
  - new_rtt = α (old_rtt) + (1 – α)(new_sample)
  - Recommended α: 0.8-0.9 (0.875 for most TCPs)
- RTO = 2 * new_rtt (i.e. TCP is conservative)

RTT Sample Ambiguity



- Karn's algorithm: ignore samples for retransmitted segments

What is Congestion?

- Load on the network is higher than capacity
  - Capacity is not uniform across networks
    - Modem vs. Cellular vs. Cable vs. Fiber Optics
  - There are multiple flows competing for bandwidth
    - Residential cable modem vs. corporate datacenter
  - Load is not uniform over time
    - 10pm, Sunday night = BitTorrent Game of Thrones

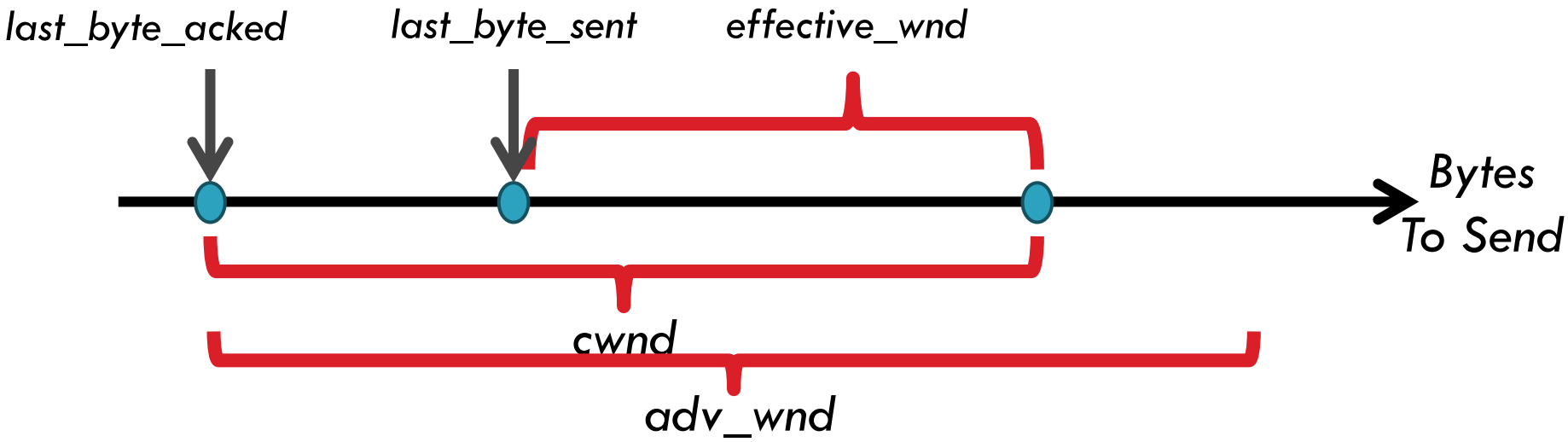
Why is Congestion Bad?

- Results in packet loss
  - Routers have finite buffers, packets must be dropped
- Practical consequences
  - Router queues build up, delay increases
  - Wasted bandwidth from retransmissions
  - Low network goodput

Congestion Window (*cwnd*)

- Limits how much data is in transit, denominated in bytes

1. *wnd* = min(*cwnd*, *adv_wnd*);
2. *effective_wnd* = *wnd* – (*last_byte_sent* – *last_byte_acked*);



Two Basic Components

1. Detect congestion
   - Packet dropping is most reliably signal
     - Delay-based methods are hard and risky
   - How do you detect packet drops? ACKs
     - Timeout after not receiving an ACK
     - Several duplicate ACKs in a row (ignore for now)
2. Rate adjustment algorithm
   - Modify *cwnd*
   - Probe for bandwidth
   - Responding to congestion

Except on wireless networks

Rate Adjustment

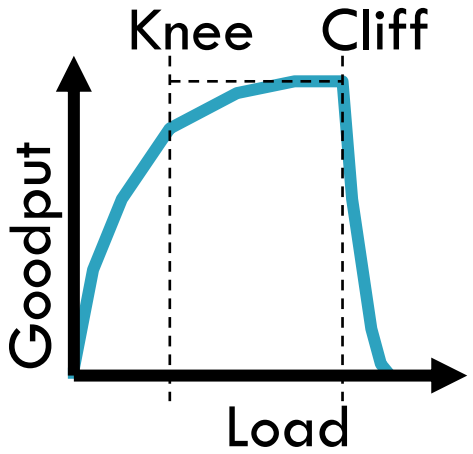
- Recall: TCP is ACK clocked
  - Congestion = delay = long wait between ACKs
  - No congestion = low delay = ACKs arrive quickly
- Basic algorithm
  - Upon receipt of ACK: increase cwnd
    - Data was delivered, perhaps we can send faster
    - *cwnd* growth is proportional to RTT
  - On loss: decrease cwnd
    - Data is being lost, there must be congestion
- Question: what increase/decrease functions to use?

Implementing Congestion Control

- Maintains three variables:
  - *cwnd*: congestion window
  - *adv_wnd*: receiver advertised window
  - *ssthresh*: threshold size (used to update *cwnd*)
- For sending, use: *wnd* = *min*(*cwnd*, *adv_wnd*)
- Two phases of congestion control
  1. Slow start (*cwnd* < *ssthresh*)
     - Probe for bottleneck bandwidth
  2. Congestion avoidance (*cwnd* >= *ssthresh*)
     - AIMD

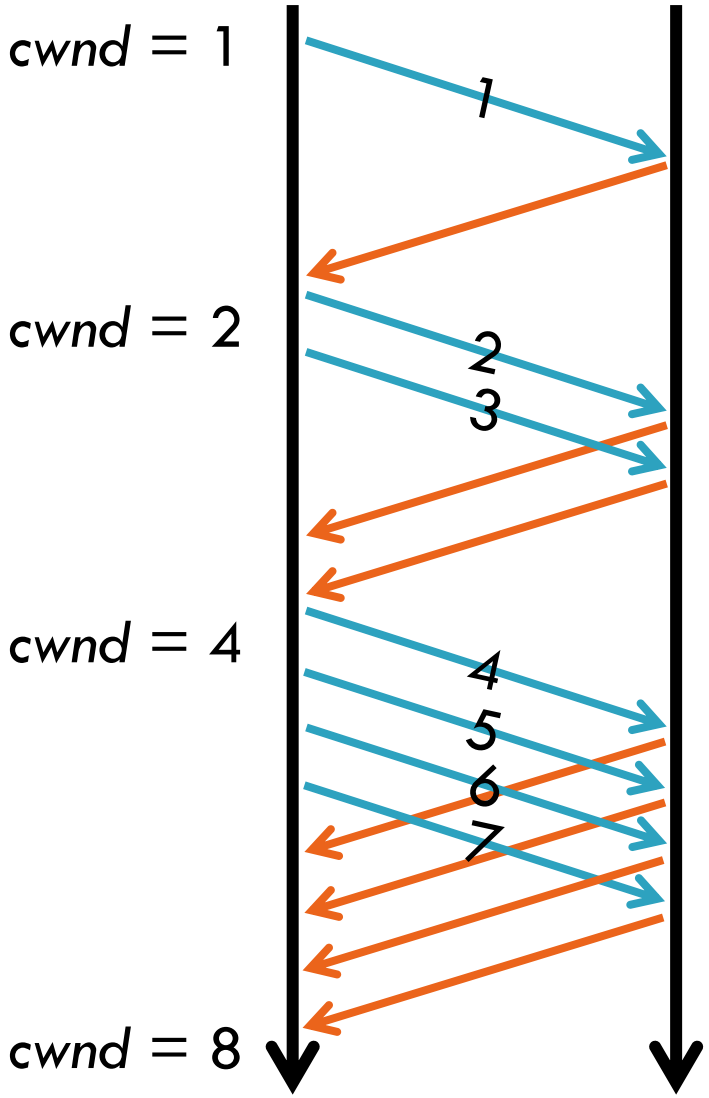
Slow Start

- Goal: reach knee quickly
- Upon starting (or restarting) a connection
  - *cwnd* =1
  - *ssthresh* = *adv_wnd*
  - Each time a segment is ACKed, *cwnd* x 2
- Continues until…
  - *ssthresh* is reached
  - Or a packet is lost
- Slow Start is not actually slow
  - *cwnd* increases exponentially



Slow Start Example

- *cwnd* grows rapidly
- Slows down when…
  - *cwnd* >= *ssthresh*
  - Or a packet drops

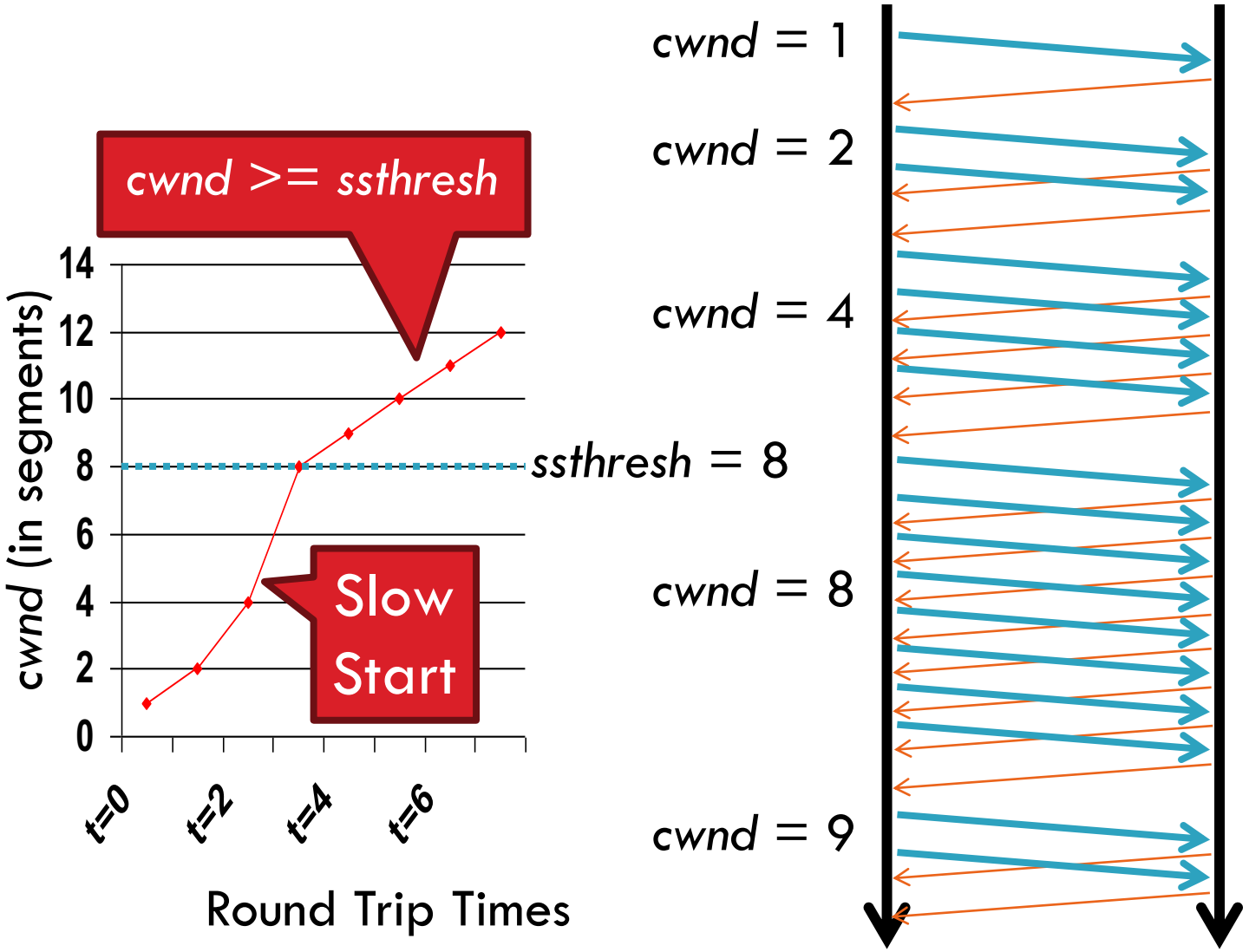


Congestion Avoidance

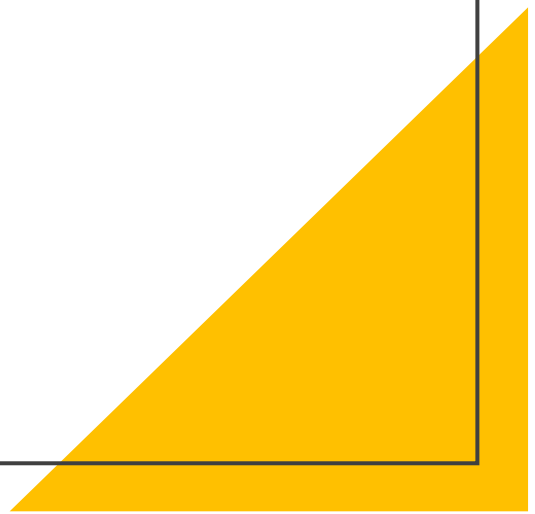
- AIMD mode
- *ssthresh* is lower-bound guess about location of the knee
- **If** *cwnd* >= *ssthresh* **then**

  each time a segment is ACKed *cwnd* += *1*/*cwnd*
- So *cwnd* is increased by one only if all segments have been acknowledged

Congestion Avoidance Example



Introduction to QUIC



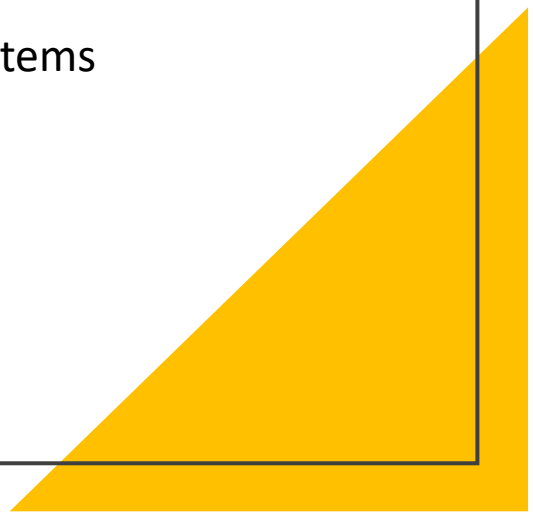
Introduction

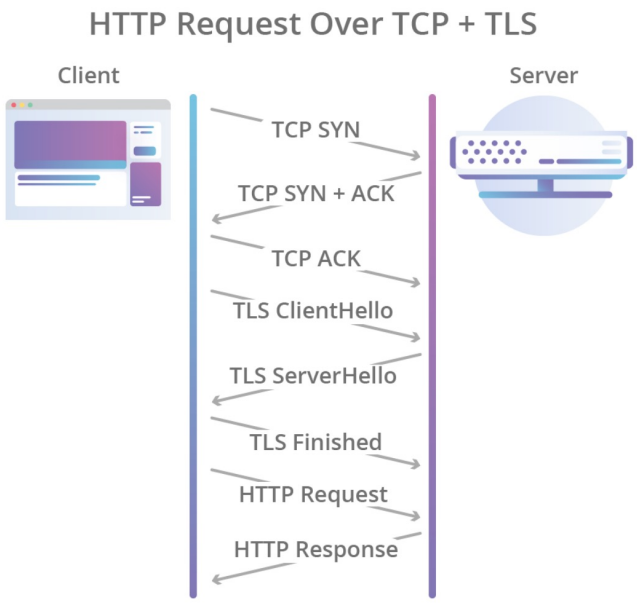
- Pronounced “quick”
- Initially proposed as Quick UDP Internet Connections
- Aim to make more fast and secure transport protocol
- In this lecture, we will focus on the “fast” aspect of QUIC

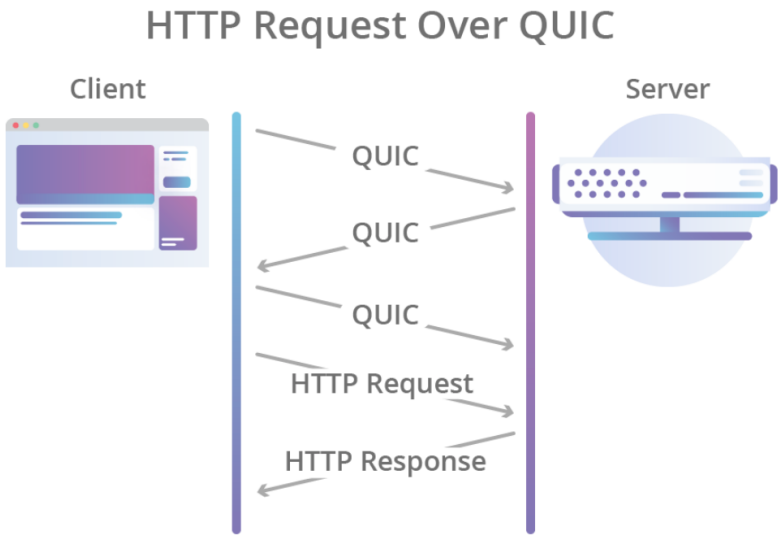
What QUIC do to make it faster than TCP

Low latency connection establishment by

- Keep cache information about previously connected systems
- Combine TCP handshake and TLS handshake



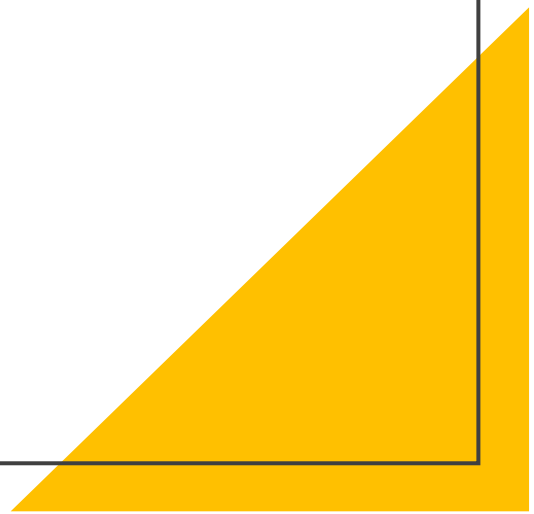

HTTP Request Over TCP + TLS
Client
Server
TCP SYN
TCP SYN + ACK
TCP ACK
TLS ClientHello
TLS ServerHello
TLS Finished
HTTP Request
HTTP Response


HTTP Request Over QUIC
Client
Server
QUIC
QUIC
QUIC
HTTP Request
HTTP Response

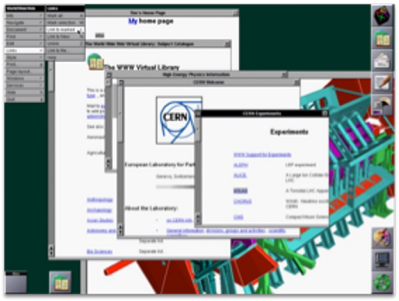
What QUIC do to make it faster than TCP

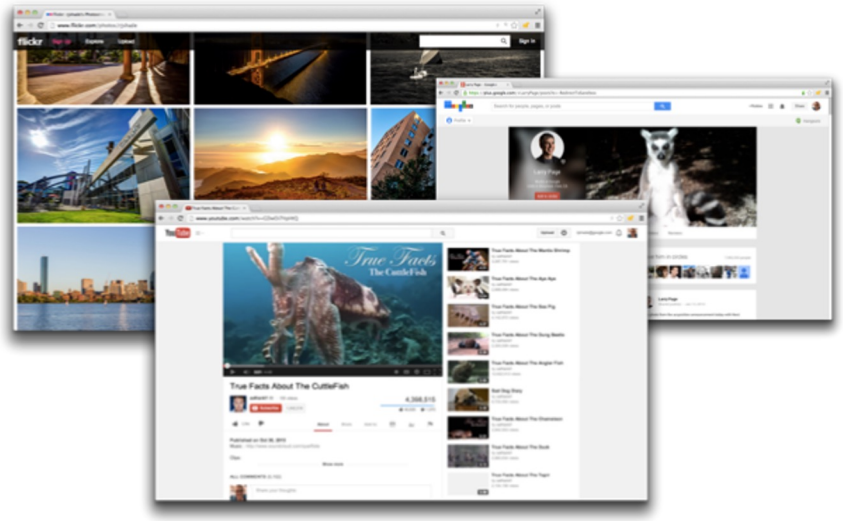
Stream multiplexing

- Can help to solve head-of-line blocking in TCP implementation (SPDY)



Google Developers Live: QUIC





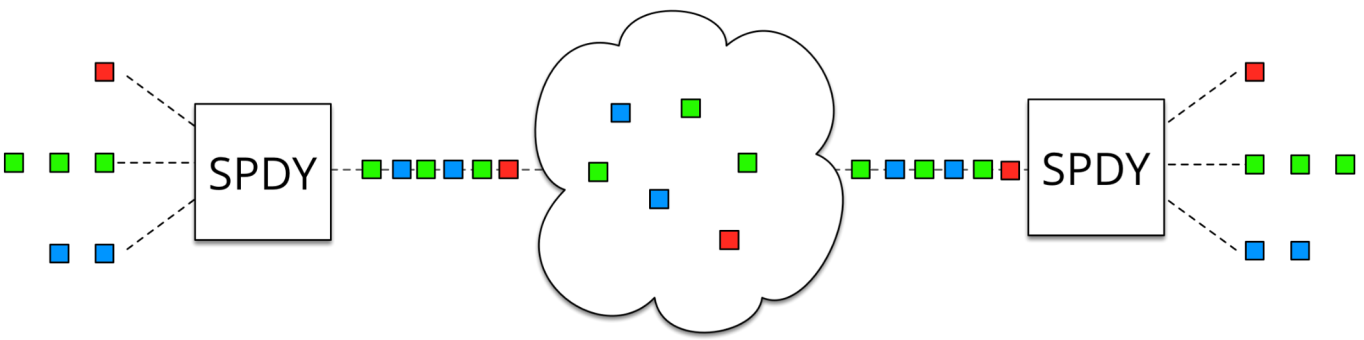
1990

single, static page
one resource
one domain

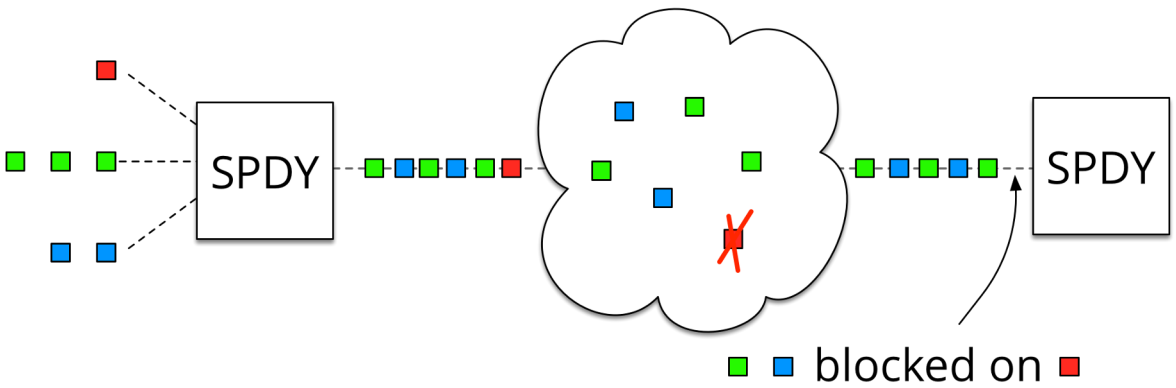
2014

1,200 KB
80 resources
30 domains

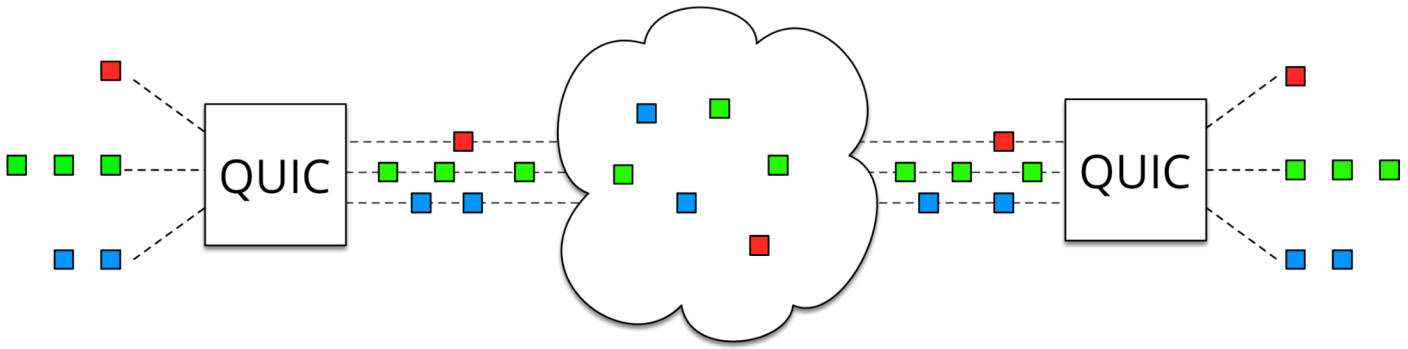
data from httparchive.org



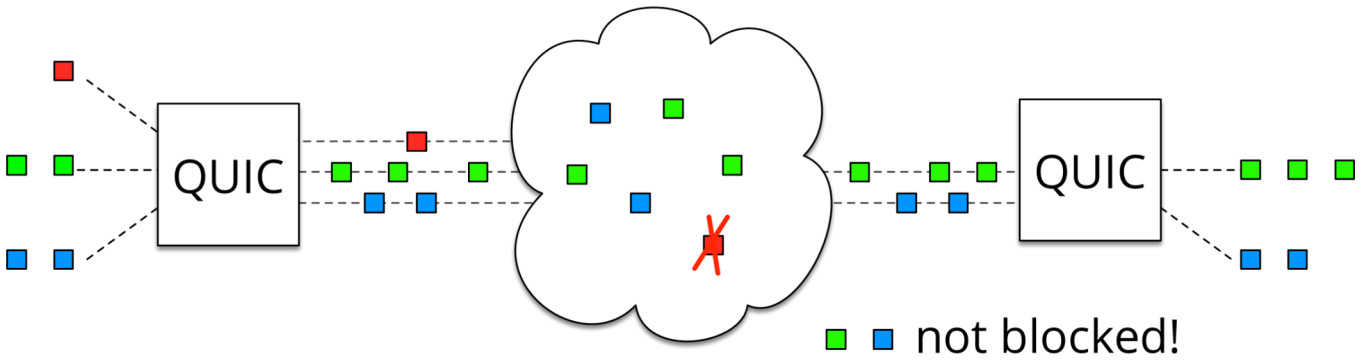
SPDY: multiplexing over TCP



Head-of-line blocking in SPDY



QUIC: multiplexing over UDP



No head-of-line blocking in QUIC!

References

- https://medium.com/@DarkDrag0nite/quic-protocol-0-rtt-connection-magic-หรือ-design-มีอยู่แค่นี้หรือมีของดีอื่นๆด้วย-9832c41ddc6a
- https://blog.cloudflare.com/the-road-to-quic/
- https://tools.ietf.org/html/draft-ietf-quic-transport-30